10주차 Google ADK Agent Programming — 섹션별 실습 코드

`10주차강의노트(Google ADK Agent Programming)_20261110_최종_v2.pptx` (142장, 목차 + Part 1~13)의 실습 코드를 폴더별 프로젝트로 정리했다. 설치·프로젝트 구조·`.env`·`adk run/web` 은 adk.dev Python 퀵스타트를 따른다. 설치된 **google-adk 2.6.3**(Python 3.11)에서 검증했다.

> 💡 **안내 문서**:
>
> - API 키 발급 및 쿼터 안내: AI Studio_API_Key.md
> - 자주 발생하는 오류 및 해결책: TROUBLESHOOTING-KR.md

개발 환경 준비 (uv 사용)

본 실습은 초고속 Python 패키지 및 가상환경 관리자인 uv를 기반으로 진행한다.

1. uv 설치

Windows

- **PowerShell (공식 권장)**:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

- **winget**:

```
winget install --id=astral-sh.uv -e
```

- **pip 사용 시**:

```
pip install uv
```

Linux / macOS

- **curl (공식 권장)**:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

- **pip 사용 시**:

```
pip install uv
```

> 설치 완료 후 터미널 창을 재시작하거나 환경 변수를 반영하여 `uv --version` 이 정상 동작하는지 확인한다.

2. Python 3.11 설치 및 가상환경 생성

ADK 2.x는 Python 3.11 이상이 필요하다. `uv` 는 시스템에 해당 버전의 Python이 없어도 자체적으로 다운로드하여 가상환경을 구성할 수 있다.

```
# Python 3.11 설치 (필요시)
uv python install 3.11

# Python 3.11 기반 가상환경 생성
uv venv --python 3.11
```

3. 가상환경 활성화

- **Windows (PowerShell)**:

```
.venv\Scripts\Activate.ps1
```

(스크립트 실행 오류 시 `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` 실행)

- **Windows (CMD)**:

```
.venv\Scripts\activate.bat
```

- **Linux / macOS**:

```
source .venv/bin/activate
```

4. 패키지 설치

```
uv pip install -r requirements.txt
```

> - `google-agents-cli` ,
> - 💡 **Tip (`uv run` 활용)**: 가상환경을 활성화하지 않고도 명령어 앞에 `uv run` 을 붙여 바로 실행할 수 있습니다. (예: `adk web --port 8008` , `uv run agents-cli --help`)

(선택) Google Cloud CLI (`gcloud`) 설치

Vertex AI 연동(방법 ②) 또는 Cloud Run 배포(part11 · part12)를 진행할 경우 시스템에 `gcloud` CLI 설치가 필요합니다. (`gcloud` 는 Python pip 패키지가 아닌 OS 설치형 바이너리 도구입니다.)

- **Windows**:

```
winget install Google.CloudSDK
```

(또는 공식 설치 프로그램 다운로드)

- **Linux / macOS**:

```
curl https://sdk.cloud.google.com | bash
exec -l $SHELL
```

- **설치 후 계정 인증 (ADC)**:

```
gcloud auth application-default login
```

5. 환경 변수 설정 (`.env`)

`.env.example` 을 `.env` 로 복사한 뒤 다음 중 **하나의 인증 방식**을 선택하여 설정한다. adk는 에이전트 폴더에서 상위 폴더로 올라가며 `.env` 를 찾으므로 루트(`ADK_Labs/.env`)에 하나만 두면 모든 Part에서 공통으로 적용된다.

방법 ①: Google AI Studio (API 키 방식, 기본 권장)

- Google AI Studio에서 API 키를 발급받아 입력한다:

```
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY=your-gemini-api-key
```

- 상세 쿼터 및 무료/유료 티어 안내는 AI Studio_API_Key.md 참고.

방법 ②: Google Cloud Vertex AI (GCP 프로젝트 연동)

Google Cloud 크레딧을 이용하거나 무료 키의 분당 쿼터(429) 제한 없이 대량 테스트를 진행하려는 경우 Vertex AI로 전환할 수 있다.

1. **GCP 프로젝트 준비 및 API 활성화**:
    - Google Cloud Console에서 프로젝트 생성
    - **Vertex AI API**(`aiplatform.googleapis.com`) 사용 설정(활성화)
2. **Google Cloud SDK(`gcloud`) 인증 (Application Default Credentials)**:
    - 로컬 터미널에서 다음 명령어를 실행하여 구글 계정으로 로그인한다:

```
gcloud auth application-default login
```

3. `.env` **설정**:
    - `GOOGLE_GENAI_USE_VERTEXAI=TRUE` 로 지정하고 프로젝트 ID와 리전을 입력한다 (`GOOGLE_API_KEY` 는 주석 처리):

```
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GOOGLE_CLOUD_PROJECT=your-gcp-project-id
GOOGLE_CLOUD_LOCATION=asia-northeast3   # 서울 리전 (또는 us-central1)
# GOOGLE_API_KEY= (주석 처리)
```

> ⚠ **MCP 호환성 주의**: MCP 예제(part06_tools)는 `mcp` 1.x가 필요하다. `mcp` 2.x가 설치돼 있으면 `McpToolset` import에서 실패한다. 문제 발생 시 `uv pip install "mcp>=1.24,<2"` 로 맞춘다.

실행 규칙

- 각 `partNN_…/` 폴더(NN = 강의노트 Part 번호)는 **에이전트들의 상위 폴더**다. 그 안의 `xxx_agent/agent.py` 가 `root_agent` 를 정의한다.
    - 대화형으로 실행: `cd partNN_…` 후 `adk web --port 8008` 또는 `adk run <에이전트폴더>`
    - 스크립트로 실행: `cd partNN_…` 후 `uv run run_*.py`
- 🟢 = LLM 없이 실행된다(키 불필요). 🔑 = Gemini 키가 필요하다.

강의노트 섹션 ↔ 실습 폴더 지도

실습 폴더 번호는 강의노트(`_20261110_최종_v2.pptx` , 142장)의 **Part 번호와 같다**: `partNN_…` = 목차(슬라이드 3)와 섹션 표지의 `Part N` . 슬라이드에 적힌 "실습 partNN …"도 이 폴더를 가리킨다. 아래 표의 Part 이름·부제·슬라이드 범위는 목차와 섹션 표지 그대로다.

강의노트 Part (표지 제목 · 부제)	슬라이드	실습 폴더	실습 내용 (괄호 = 슬라이드)	스크립트
Part 1 · 01. 프레임워크 지형과 ADK 5대 프레임워크 비교, 아키텍처 패러다임, 선택 기준	5–15	`part01_frameworks/`	개념 강의 (실습 코드 없음, 폴더에 안내 README만)	—
Part 2 · 02. Google ADK 아키텍처 Runner · Event · Session · Memory로 보는 내부 엔진	16–25	`part02_architecture/`	`chat_time_weather_agent` (실시간 시간·날씨·대화) · Event 생명주기(21–24) · `memory_layers_agent` (이중 계층 메모리, 25)	🔑 `run_events.` `run_workflow_e` `run_memory_lay`
Part 3 · 03. 개발 환경과 CLI adk create · run · web · api_server와 agents-cli	26–43	`part03_cli/`	`my_agent` (adk create 구조) · 직접 Runner 코드(27–29·33) · adk 내부 재현(30–32) · adk 명령(34–37, api_server 37) · agents-cli 개요(38–43)는 `part12_agents_cli/`	`README-KR.md` · 🟢 / 🔑 `adk_int`
Part 4 · 04. LLM Agent 프로그래밍 Agent 파라미터, 지시문, 구조화 출력, 모델, Runner	44–51	`part04_llm_agent/`	weather(45) · tutor(State 템플릿, 47) · reviewer(output_schema, 48) · litellm(49) · planner(50) · Runner(51)	`run_with_ru` `[weather|tutor|`
Part 5 · 05. Skills for ADK Agents SKILL.md · 점진적 공개 · SkillToolset · google.adk.skills · Agents CLI 스킬	52–69	`part05_skills/`	`skills_agent` (SkillToolset, 58–62) · `skills_explorer.py` (google.adk.skills 로드·검증·도구 노출, 59–62 · 코딩 에이전트 스킬 57·67) · 코딩 에이전트 스킬 설치(65–68)는 `part12_agents_cli/coding_with_ai/`	`skills_expl` `[load|validate|` `run_skills.py`
Part 6 · 06. Tools: 에이전트의 손과 발 Function · 내장 도구 · AgentTool · MCP · OpenAPI · 장기 실행	70–77	`part06_tools/`	shop(Function Tool + 확인, 72) · analyst(AgentTool · google_search · 코드실행, 74) · mcp(75) · openapi(HR API + 목업 서버, 76) · refund(LongRunning, 77)	`run_tools.p` `[shop|analyst|m`
Part 7 · 07. Session · State · Memory · Callback 에이전트가 기억하고, 상태를 넘기고, 스스로를 감시하는 방법	78–84	`part07_context/`	state_agent(접두사 · SQLite 세션, 80–81) · memory_agent(82) · guarded_agent(콜백, 83–84) · AuditPlugin(84)	🔑 `run_state_s` `run_memory.py`
Part 8 · 08. 멀티 에이전트 구성 sub_agents 위임, 1.x Workflow Agent, 2.0 협업 모드	85–90	`part08_multi_agent/`	help_desk(sub_agents transfer, 87) · report_pipeline(1.x Sequential/Parallel/Loop, 88) · travel_planner(2.0 coordinator, 89–90)	`run_multi_a` `[help_desk|pipe`
Part 9 · 09. ADK 2.0 Graph Workflow Node · Edge · Route · JoinNode로 흐름을 코드로 고정한다	91–100	`part09_graph_workflow/`	seq(95) · route(96) · fan(JoinNode, 97) · loop(back-edge, 98) · refund(사례 ①, 99) · expense(사례 ②, 100)	🟢 `run_workflo`
Part 10 · 10. Dynamic · HITL · Retry 코드로 오케스트레이션하고, 사람을 기다리고, 실패에 견딘다	101–109	`part10_dynamic_hitl/`	dynamic(102) · dynamic_parallel(103) · hitl · feedback(105) · approval_gate(106) · retry(107)	🟢 `run_part10.`
Part 11 · 11. 평가 · 배포 · 운영 adk eval, Cloud Run, Agent Runtime, 트리거, A2A, 관측성	110–124	`part11_eval_deploy/`	evalset · AgentEvaluator(111) · 서버와 `adk deploy cloud_run` · `gcloud run deploy` + 컨테이너(120) · Agent Runtime 배포 경로 adk/agents-cli/SDK(121) · Pub/Sub 트리거(122) · A2A(123)	`make_evalset.p` `test_container` `run_server_sta` `deploy/deploy_` `DRY_RUN=1 bash` 🔑 `run_a2a_cli`
Part 12 · 12. agents-cli 엔드투엔드 워크플로우 실제 프로젝트 해부: create · playground · eval · deploy · run	125–137	`part12_agents_cli/`	프로젝트 해부(126) · 개발 6단계(127) · Coding with AI(128–129, `coding_with_ai/`) · 한 서버 두 입구(130–131) · eval(132–133) · deploy · infra(134–135) · 호출(136) · 실습(137)	`adk|a2a`
Part 13 · 13. 실습과 종합 오늘 만든 조각을 하나의 에이전트로 조립한다	138–142	`part13_lab/`	학사 문의 에이전트 1~6단계(139): 기본 → 도구 → 가드레일 → Route 워크플로우 → HITL+Retry → eval/deploy	`run_lab.py [1.`

`common/`에는 공통 헬퍼가 있다. `.env` 로딩과 Runner 래퍼(`Session.send` / `Session.resume_input` : HITL 재개)를 제공한다.

자주 쓰는 명령

```
cd part09_graph_workflow && adk web --port 8000
```

```
cd part11_eval_deploy && adk eval weather_agent weather_agent/weather.evalset.json --config_file_path
weather_agent/test_config.json --print_detailed_results
```

`adk eval` 실행 방법

`adk` 명령이 다른 Python 설치에 연결돼 있으면 `adk eval`이 `Eval module is not installed`를 내고 실패할 수 있다. 예를 들어 `D:\Python311\Scripts\adk.exe`가 `C:\Users\...\Python311\python.exe`를 가리키는 경우다. 이때는 `adk` 대신 `python -m google.adk.cli` 또는 `uv run adk`로 실행한다. 인자는 같다.

part11 날씨 에이전트 평가 (`part11_eval_deploy` 폴더에서):

```
adk eval weather_agent weather_agent/weather.evalset.json --config_file_path
weather_agent/test_config.json --print_detailed_results
# 또는: python -m google.adk.cli eval weather_agent weather_agent/weather.evalset.json --
config_file_path weather_agent/test_config.json --print_detailed_results
```

part13_lab 학사 에이전트 평가 (`part13_lab` 폴더에서, 결과 예: `Tests passed: 3`):

```
adk eval step3_callback step6_eval_deploy/academic.evalset.json --config_file_path
step6_eval_deploy/test_config.json
# 또는: python -m google.adk.cli eval step3_callback step6_eval_deploy/academic.evalset.json --
config_file_path step6_eval_deploy/test_config.json
```

`uv pip install -r requirements.txt` `adk web`, `adk run`, `adk api_server` 실행할 수 있다 (예: `adk web --port 8000` 또는 `python -m google.adk.cli web --port 8000`).

```
cd part11_eval_deploy/a2a_demo && uv run uvicorn prime_agent.agent:a2a_app --port 8001
```

```
cd part06_tools && uv run openapi_agent/hr_mock_server.py
```

HITL 재개 방식 (part09 · part10 · part13)

`RequestInput`이 실행되면 워크플로우가 `adk_request_input` function_call 이벤트를 내고 멈춘다. 재개하려면 `FunctionResponse(name="adk_request_input", response=...)`를 `common/runner_util.py`의 `Session.resume_input()`에 넘겨 처리한다. `adk web`에서는 UI 입력창에서 바로 응답할 수 있다.

무료 키 사용 시 주의

- Gemini 무료 등급은 **모델마다 분당 5회·하루 20회 정도**로 제한된다. 에이전트 한 턴에 LLM 호출이 2~4회 생긴다.
- `common/bootstrap.py`가 429·5xx 응답을 10s→20s→40s 간격으로 자동 재시도한다(`run_*.py` 실행 시에만 적용, 끄려면 `ADK_LAB_RETRY=0`). 하루 한도를 넘긴 경우는 재시도로 풀리지 않으니 `.env`의 `ADK_MODEL`을 다른 모델로 바꾼다.
- `google_search`(part06 analyst의 searcher)는 이 무료 키에서 429로 막혔다. 결제가 연결된 키나 Vertex AI에서 실행한다.
- `adk` 명령이 다른 Python에 연결돼 있으면 `adk eval`이 "Eval module is not installed"를 낼 수 있다. 이 때는 `python -m google.adk.cli eval ...`처럼 실행한다(`adk web`, `adk run`도 같은 방식으로 실행할 수 있다).

검증 결과 (google-adk 2.6.3)

키 없이 실행한 결과:

- 🟢 표시 스크립트(part02 · part09 · part10 · part13의 4·5단계, part07 가드레일 차단)가 슬라이드의 기대 출력과 같다. part02 `run_memory_layers.py offline` : 새 세션에 `user:name` 만 남고 `trip` 은 없음, InMemory 세션은 재시작 시 소멸 · SQLite 세션은 보존, `search_memory('부산')` 1건 · `'부산에서'` 0건. 예: `auto-approved $42` , `joined={...}` , `sum of squares=14` , `3회차 성공` .
- 에이전트 패키지 38개가 모두 import되고 root_agent가 정상 생성된다.
- 🟢 `part05_skills/skills_explorer.py` : L1 목록 · L2/L3 로드 · `<available_skills>` XML, 잘못된 스킬 4종이 로드 시점에 실패, 활성화 후 `get_weather` 노출, 코딩 에이전트 스킬 7종(google-agents-cli-*)을 ADK 로더로 읽음.
- 🟢 `part11_eval_deploy/tests/test_container_app.py` : gcloud 배포용 컨테이너 서버(main.py)가 `/list-apps` 에 weather_agent를 내고, 세션을 `SESSION_SERVICE_URI` DB에 저장(2 passed). `deploy_agent_runtime_sdk.py` (dry-run): AdkApp 생성·SDK 배포 설정 출력.
- 🟢 `part11_eval_deploy/run_server_state.py` : 메모리 세션은 인스턴스가 바뀌면 유실 · 공유 DB 세션은 유지, Cloud Run(`K_SERVICE`) 감지 시 InMemorySessionService 선택.

Gemini 무료 키로 실행한 결과(2026-11-09, `gemini-3.5-flash` / `gemini-3.5-flash-lite`):

실습 폴더	결과
part02	`get_current_time` 호출 → 응답, 이벤트 4개 기록 · `memory_layers_agent` : 세션 2에서 이름은 `user:` State로, "매운 음식 못 먹음"은 `preload_memory` 로 회상
part03	`run_direct_runner.py` : function_call → function_response → 최종 응답 · `adk_internals.py` : 기본 세션 = `my_agent/.adk/session.db` (PerAgentDatabaseSessionService), `--in_memory` → InMemory, `--ask` 한 턴 응답
part04	weather · tutor(State 템플릿) · reviewer(`state['review']` 구조화 출력) · planner(thought 출력) · litellm(Gemini fallback) 모두 동작
part05	skills(list_skills → load_skill → get_weather → run_skill_script → load_skill_resource, 폭염·조회 실패 예외 규칙 적용) 동작
part06	shop(`adk_request_confirmation` 승인 후 결제) · calculator 코드 실행 · MCP(mcp 1.30 + Node) · OpenAPI(목업 서버) · refund(LongRunning 재개) 동작. **google_search만 무료 키 한도로 실패**
part07	user:/app: 접두사가 새 세션에서도 유지됨 · Memory 회상 · PII 차단 / 수량 검증 / AuditPlugin
part08	`BK-KE1201-W` ,
part09	모든 워크플로우와 LLM 노드(환불 안내문, 경비 검토, seq) 동작
part11	`pytest tests` 2 passed · A2A 원격 `prime_agent` 위임 동작
part12	`pytest tests` 9 passed · agents-cli 1.13.0 `run --mode adk/a2a` 동작 · `agents-cli eval run` 4건 평균 5.0
part13	1~5단계 동작 · `adk eval` academic 3/3 통과